

Q247: LC 21 - Merge 2 sorted lists

Authored by: Arpit Raj, LNMIIT Jaipur | July 15, 2026

Problem Overview

1 → 2 → 4

1 → 3 → 4

then o/p: 1 → 1 → 2 → 3 → 4 → 4

Asked in:

Epam

Infosys

TCS

Atlassian

Arista

TC = $O(m+n)$

SC = $O(1)$

Algorithm Steps

- S1** decide who will become the head
- S2** make a curr and make it equal to head
- S3** pick the smaller ones as curr → next
- S4** after finishing check if some elements are left to be added, one of 2 lists might still have elements waiting to be added

Standard Template

```
while (A != NULL && B != NULL) {  
    if(A->val < B->val) {  
        take A  
        A = A->next  
    }  
    else {  
        take B  
        B = B->next  
    }  
}  
and then attach remaining elements
```

C++ Implementation

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {

        if(list1==NULL) return list2;
        if(list2==NULL) return list1;

        ListNode* head;
        if(list1->val < list2->val){
            head=list1;
            list1=list1->next;
        }
        else{
            head=list2;
            list2=list2->next;
        }

        ListNode* curr=head;
        while(list1!=NULL && list2!=NULL){
            if(list1->val < list2->val){
                curr->next=list1;
                list1=list1->next;
            }
            else{
                curr->next=list2;
                list2=list2->next;
            }
            curr=curr->next;
        }

        if(list1!=NULL){
            curr->next=list1;
        }
        else{
            curr->next=list2;
        }

        return head;
    }
};

```